

these plots show, even in these more challenging cases, we observe non-trivial entity tracking abilities. However, as the number of operations grows, performance rapidly approaches the random baseline, suggesting that entity tracking becomes increasingly more brittle as more state changes need to be considered jointly.

4.5 Discussion

Our results show that among the models we evaluated, only GPT-3.5 text-davinci-003 exhibit non-trivial entity tracking behavior. While its performance does decrease as the number of operations increases, the model still produced many accurate predictions even after six or seven sequences of operations. Furthermore, through the AltForms experiment with low lexical overlap between demonstration/test, we also ruled out the possibility that the demonstrations are teaching the model this task or that the model is primarily relying on superficial slot-filling heuristics. Therefore, we conclude that text-davinci-003 does have some capacity to track discourse entities in linguistically expressed contexts. To a lesser extent, we also observed this capacity in the highly context-dependent AmbiRef and MoveContents experiments. This provides further evidence against superficial heuristics in the performance we observe; at the same time, this highlights scenarios in which even the most recent models exhibit difficulties.

On the other hand, entity tracking behavior did not surface in GPT-3 davinci (likely of similar size as GPT-3.5 text-davinci-003), a model pre-trained primarily on text corpora on the next word prediction objective. This was also true for denoising models that have been finetuned on many tasks combined with instructions and demonstrations: the Flan-T5 models also showed near-zero accuracy on non-trivial examples.

Our results overall show that there exists a language model that can perform entity tracking to some degree, but this capacity does not necessarily surface in all sufficiently large models trained on large corpora. Then, which factors are responsible for this difference? Given that davinci and text-davinci-003 differ along at least two dimensions (text-davinci-003 is based on a model that was trained on code and it was trained with additional human feedback (Ouyang et al., 2022); see Table 1), our initial results do not shed light on what exactly contributes to this difference. We therefore

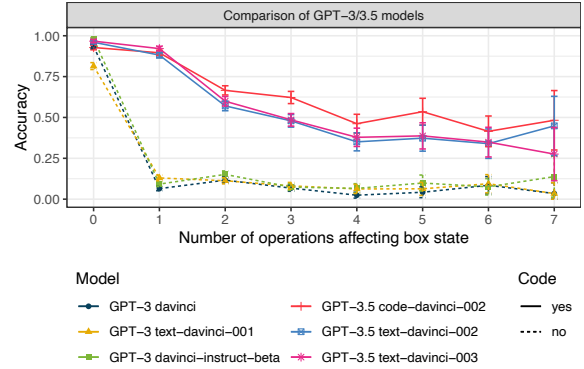


Figure 5: Accuracy on state prediction for different GPT-3 models. Solid lines denote models trained on code and text, and dotted lines denote models mainly trained on text.

conducted a follow-up experiment where we compared a range of GPT-3 and GPT-3.5 models to identify factors that contribute to the stark difference between davinci and text-davinci-003.¹¹

Training on Code Encourages Entity Tracking Behavior As Table 1 shows, two key dimensions of variation across models are additional training on human feedback and pretraining on code. If additional training on human feedback imbues language models with the ability to track entities, all models except for GPT-3 davinci and GPT-3.5 code-davinci-002 should be able to track entities. If, on the other hand, pretraining on code leads to better entity tracking, we expect all GPT-3.5 models to outperform GPT-3 on our task. As Figure 5 shows, GPT-3.5 models that have been trained on code systematically outperformed GPT-3 models, including code-davinci-002 that was not trained on human feedback. This suggests that a substantial representation of code in the pretraining data is beneficial for a language model’s entity tracking capacity to surface.

A further question that our results so far do not answer is to what extent model size matters and whether models at the scale of Flan-T5 can also exhibit non-trivial entity tracking behavior. Since there exist no smaller models that have been trained with the same objective and training data as the GPT-3.5 models, we explore this question through finetuning experiments with T5.

¹¹To limit inference costs, we used the same subsample of data as in the AltForms experiment.

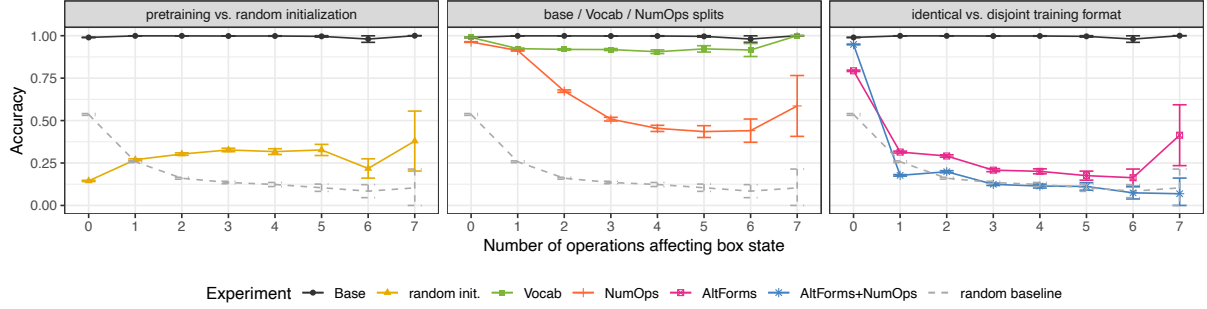


Figure 6: Results for finetuned T5 models.

5 Experiment 2: Finetuning

We investigated whether smaller models at the scale of T5 can *learn* to track entity states through a series of experiments where we provide supervised training to the models.

5.1 Train/test splits

As discussed in Section 3.1, one challenge of evaluating entity tracking abilities is distinguishing this capacity from simple heuristics such as templatic slot-filling. We therefore designed various types of training/evaluation mismatches that block several possible shortcuts as described below.

Base Split Here, we used the same format for training and evaluation examples. All initial states differed across training and evaluation to block simple slot-filling heuristics, as discussed in Section 3.2.

NumOps Split The NumOps split restricts the maximum number of operations within a single example in the training set to 2, but includes up to 12 operations in the evaluation set. This split was intended to test whether a finetuned model is able to generalize to longer sequences of operations than it has seen during finetuning.

Vocab/AltForms Splits The vocab split tests whether objects that are not part of the set of objects used during training can also be adequately tracked. We compiled a list of comparatively infrequent object names (e.g., *pomelo*, *furby*, *Flav-R-Straw*; not in BNC) and sampled the training and test sets using two completely disjoint sets of object names. The training set used the infrequent object list and the test set used the original object list. The AltForm split follows the design described in Section 4.2. These splits aim to tease apart whether the model learns to associate specific words/phrases with the operations or whether finetuning leads to more generalizable entity tracking behavior.

AmbiRef/MoveContents Splits The AmbiRef and

MoveContents splits follow the design described in Section 4.2. These splits aim to test whether the model can learn to interpret operations that are underspecified without considering the current state of the affected boxes. For these splits, the training and test examples share the same format.

5.2 Models

We evaluated T5-base, the best-performing model in Li et al. (2021), by finetuning it on each of the datasets described above. As an additional baseline, we compared against T5 with randomly initialized parameters trained directly on our datasets.

5.3 Results and Discussion

Pretrained T5 can Learn to Perform Entity Tracking

As shown in Figure 6 (left), finetuning T5 leads to near-perfect accuracy on the base split. This suggests that the model is capable of learning this task. Training a randomly initialized T5 did not yield the same result: the accuracy of a model trained from random weights is considerably lower, due to the model almost exclusively predicting that a box is empty. These two results suggest that pre-training is crucial for the model to be able to learn this task. Furthermore, the model’s entity tracking capacity is robust to novel object names at test time, with only minor degradation on accuracy (Figure 6, middle). Training only on operation sequences with a maximum length of 2 (NumOps split) leads to a larger degradation in performance for longer operation sequences, but even for longer operation sequences, the model is able to infer the correct final state in more than 45% of the cases. Finally, the model performance does degrade substantially when the training examples have low lexical overlap with test examples (Figure 6, right). Nevertheless, model performance remains above the random baseline when the model is trained on up to 12 operations (pink line). If we trained only on up to two

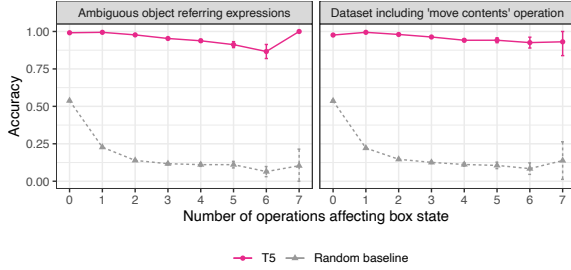


Figure 7: Accuracy of finetuned T5 for AmbiRef (left) and MoveContents (right) datasets.

operations (blue line), however, the performance degradation was compounded and performance no longer exceeded the random baseline. These results suggest that finetuning on an entity tracking task does lead to entity tracking abilities that generalize to many challenging scenarios. At the same time, however, performance rapidly degrades as the finetuning and evaluation splits become increasingly dissimilar, and it remains an open question to what extent finetuning on a limited domain such as the boxes environment transfers to more general entity tracking abilities in more naturalistic discourse.

Interpreting Context-Dependent Operations Remains Challenging If we include operations that either contain ambiguous referring expressions (Figure 7, left) or a *Move contents* operation (Figure 7, right) in the finetuning and evaluation data, we see a slight degradation in performance and the model no longer achieves near-perfect accuracy, despite training and evaluating on examples of the same format. In both cases, the model almost exclusively made mistakes with examples that require the interpretation of context-dependent operations. This suggests that the context-dependent operations do compound the difficulty of entity tracking even when the models receive direct training signal.

6 Conclusion

We set out to investigate whether pretrained LLMs exhibit entity tracking behavior. We developed a task that allowed us to evaluate whether LLMs can predict the state of an entity based on an initial state description and operations that act upon it. In the first set of experiments, we found that GPT-3 davinci, a vanilla pretrained language model, and Flan-T5, an instruction-finetuned language model, fail at this task and simply repeat the initial state description. The GPT-3.5 models (the core difference with the aforementioned models being code in pretraining corpora), on the other hand, exhib-

ited non-trivial entity tracking behavior. Even after many operations affecting an entity state, they consistently performed above a strong random baseline. In the second set of experiments, we showed that this behavior can also to a large extent be learned by smaller models such as T5. When we finetune the model on this task, it is also able to perform entity tracking on examples that differ along several dimensions from the training data. Taken together, our results provide evidence that (a) vanilla language models that have mainly been trained on text data do not exhibit entity tracking abilities out of the box, (b) pretraining on code and text data considerably improves this ability,¹² and (c) finetuning on this task can make this behavior also surface in smaller models that have primarily been trained on text data, although it remains an open question how general this ability is in smaller finetuned models.

What are reasons behind the efficacy of training both on text and code? For producing correct code, tracking the states of variables is important. Therefore, to speculate, this kind of pretraining data may provide a stronger signal for the model to track entities compared to pure text data. It could also be that, as previously speculated (Potts, 2020; Merrill et al., 2021, i.a.), pretraining on code provides additional grounding.

The present results also highlight the importance of the composition of the pretraining data. Our results showed that models that are trained on the language modeling objective on large corpora show dramatically different behavior on entity tracking depending on whether the training data includes substantial amount of code or not. In this light, future work could investigate the effect of pretraining on code more widely, including its effect on the model representations and to what extent it facilitates the emergence of world models in LMs.

Finally, we also laid out several principles that should be followed when evaluating state tracking abilities. Apart from these specific principles, we make a more general point that in assessing abilities related to meaning, one needs to consider potential strategies that the model could use to solve the task and make sure that the test examples do not mimic the distributional patterns of the training or finetuning data. Only then can we properly assess

¹²This is also compatible with the observation by Sap et al. (2022) that GPT-3.5 performs considerably better than GPT-3 in answering factual questions about object location changes. Further, see Madaan et al. (2022) for other tasks for which pretraining on code seems to be beneficial.